ABM Account Intelligence Platform — V1 Vision

Owner: Sunny Dsouza • **Primary user:** Galyna • **Status:** Vision locked, ready to design + build

---

TL;DR

A **single-tenant internal web app** that ingests target accounts and signals from Definitive, Apollo, Apify, Crunchbase, and Google News; scores them with Claude Opus 4.7 using Magical's three ABM frameworks (Specialties, Payers, Health Systems); surfaces *what's new this week* to Galyna in a triage view; and produces per-account one-pager PDFs with editable tier overrides. Re-scoring auto-fires for Tier 1/2 accounts when new signals land. No outbound automation in V1 — activation is a status flag only.

V1 in one sentence

> Galyna opens a web app on Monday morning, sees a triage feed of what changed across her ~500 target accounts, opens any of them to a structured one-pager (scored by Claude with live web research), downloads the PDF, optionally overrides the tier with a note, and marks the account as activated. The system auto-re-scores Tier 1/2 accounts when fresh signals arrive in the background.

Who & why

Galyna — ABM analyst at Magical. Today she runs scoring manually in browser-based Claude sessions, copy-pastes results into Notion, maintains target lists in Google Sheets, and cross-references signals from 4–5 separate tools by hand. She does this for 50–200 accounts a week.

The pain V1 solves: 1. **Repetitive scoring** — Claude-in-browser is slow and unaudited. V1 makes scoring a button-press with a structured, queryable record. 2. **Signal blindness** — leadership changes, funding rounds, EHR migrations happen across Apollo / LinkedIn / Crunchbase / news; today she only catches them ad-hoc. V1 streams them into one triage view. 3. **No memory** — every score is fresh; she can't see "OrthoIndy was 21/30 in Q3, now 25/30." V1 keeps history. 4. **Manual list curation** — she maintains the target universe in Google Sheets. V1 makes the DB the source of truth, with Definitive feeding it.

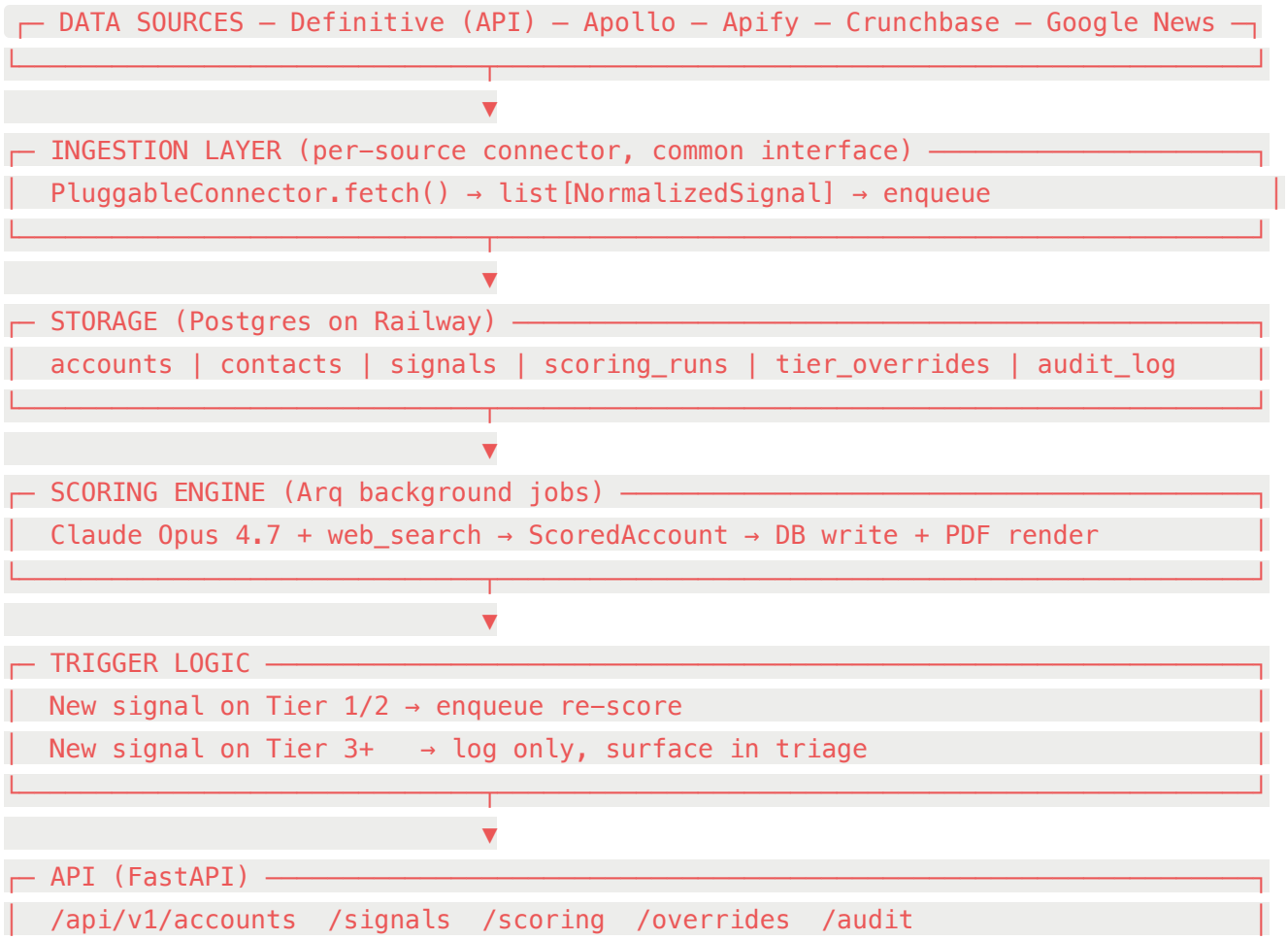
The pain V1 explicitly does NOT solve (yet): Outbound activation, Salesforce sync, mobile, multi-user, deck/portfolio reports, weekly digest emails. Those land in V2+.

Non-goals (V1)

- ❌ Outbound email / LinkedIn sequence automation
- ❌ Notion auto-push (app is the source of truth)
- ❌ Salesforce / SFDC sync
- ❌ Multi-user / role-based access (Clerk auth ready for V2)
- ❌ Snowflake / Omni dashboard
- ❌ Mobile UI (desktop-first)
- ❌ Multi-segment accounts (one segment per account, strict)
- ❌ Weekly digest emails
- ❌ Custom outbound playbooks per segment
- ❌ Public API

Explicit non-goals exist to keep V1 shippable and to give Galyna's feedback a place to land before we commit to V2 features.

Architecture at a glance



Tech stack — every box, opinionated

Layer	Tech	Why
Frontend	Next.js 15 (App Router) + TypeScript + Tailwind + shadcn/ui	Industry default; pairs with Claude Design output cleanly; great DX
Backend API	FastAPI + Pydantic v2	Already the scoring code's home; async-native; great OpenAPI
Database	Postgres 16 (Railway-managed)	Relational fit, JSON support for payloads, free migration path to Snowflake
Migrations	Alembic	The Pydantic ecosystem standard
Cache + Queue	Redis (Railway-managed)	Backs Arq jobs; future feature-flag store
Background jobs	**Arq** (NOT Celery)	Async-native, Pydantic-friendly, ~1/10th the boilerplate of Celery
Auth	Clerk (magic link + Google SSO)	30-min wire-up; multi-user-ready when V2 lands
LLM	Anthropic Claude Opus 4.7 + `web_search_20260209`	Already validated for scoring quality
Hosting	All on Railway (single project, multi-service)	One vendor, one bill, one deploy pipeline
Observability	Sentry (errors) + Railway logs (JSON structured)	Free tier covers V1 scale
Python tooling	`uv` (package mgr) + `ruff` (lint/format) + `mypy` (types) + `pytest` (tests)	Modern Astral stack; 10× faster than pip/black/flake8
JS tooling	`pnpm` + `vitest` + `playwright` (smoke only) + `biome` (lint+format)	Faster than npm + eslint; one binary for lint+format

Layer	Tech	Why
CI	GitHub Actions	Matrix: lint, type-check, test, build, on every PR; auto-deploy main → Railway

Why these specific picks (the non-obvious ones)

- **Arq over Celery** — Celery is heavyweight, sync-first, and Galyna's traffic is 100–500 jobs/week. Arq is async-native (matches FastAPI), runs in one process, has clean retry semantics, and reads Pydantic models natively. Ship in 1 day vs. Celery's 3.
- **Postgres over SQLite** — even at 1,000 accounts SQLite works, but Postgres on Railway is free at this tier, gives you concurrent writes (matters once background jobs land), and removes the migration question for V2.
- **shadcn/ui over MUI/Chakra** — copy-paste components, no runtime dependency, Tailwind-native. Pairs cleanly with whatever Claude Design produces.
- **Clerk over NextAuth/Supabase Auth** — magic link + Google SSO in 30 minutes. NextAuth is free but you maintain it. Supabase Auth requires Supabase. Clerk's free tier covers Galyna for a year.
- **uv over poetry/pip** — Astral tooling is rewriting Python's package story. `uv` is 10–100× faster, lockfile-first, Python-version aware. Same family as `ruff`.
- **biome over eslint+prettier** — one binary, 30× faster, sane defaults. The Rust toolchain is winning.

Data model

Core entities (Phase 1)

```
-- A company we're targeting
CREATE TABLE accounts (
  id              BIGSERIAL PRIMARY KEY,
  name            TEXT NOT NULL,
  segment         TEXT NOT NULL CHECK (segment IN ('specialties','payer','hs')),
  domain          TEXT,
  is_activated    BOOLEAN NOT NULL DEFAULT FALSE,
  current_score   NUMERIC(5,2),
  current_tier    TEXT,
  current_max     INTEGER,
  last_scored_at  TIMESTAMPTZ,
  first_seen_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  source          TEXT NOT NULL,  -- 'definitive' | 'manual' | 'apollo' | ...
  UNIQUE(name, segment)
);
```

```sql
-- A person at an account
CREATE TABLE contacts (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  name            TEXT NOT NULL,
  role            TEXT NOT NULL,
  email           TEXT,
  linkedin_url    TEXT,
  is_primary      BOOLEAN NOT NULL DEFAULT FALSE,
  source          TEXT NOT NULL,
  observed_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- An observed event. Drives re-scoring + triage feed.
CREATE TABLE signals (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  signal_type     TEXT NOT NULL,    -- leadership_change | funding_round | ...
  source          TEXT NOT NULL,    -- definitive | apollo | apify_linkedin | ...
  title           TEXT NOT NULL,
  payload         JSONB NOT NULL,
  url             TEXT,
  observed_at     TIMESTAMPTZ NOT NULL,
  processed_at    TIMESTAMPTZ,      -- when re-scoring fired (NULL = pending or skipped)
  fingerprint     TEXT NOT NULL,    -- dedupe key: hash(source,type,url,observed_date)
  UNIQUE (account_id, fingerprint)
);

-- One scoring execution
CREATE TABLE scoring_runs (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  total_score     NUMERIC(5,2) NOT NULL,
  max_score       INTEGER NOT NULL,
  tier            TEXT NOT NULL,
  raw_markdown    TEXT NOT NULL,
  structured_json JSONB NOT NULL,
  model           TEXT NOT NULL,
  cost_usd        NUMERIC(8,4),
  input_tokens    INTEGER,
  output_tokens   INTEGER,
```

```
  stop_reason     TEXT,
  parse_failed    BOOLEAN NOT NULL DEFAULT FALSE,
  triggered_by    TEXT NOT NULL,    -- 'manual' | 'signal' | 'scheduled'
  triggering_signal_id BIGINT REFERENCES signals(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Galyna's tier overrides (auditable)
CREATE TABLE tier_overrides (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  scoring_run_id  BIGINT NOT NULL REFERENCES scoring_runs(id),
  original_tier   TEXT NOT NULL,
  override_tier   TEXT NOT NULL,
  reason          TEXT NOT NULL,
  set_by_user_id  TEXT NOT NULL,    -- Clerk user ID
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Append-only audit log
CREATE TABLE audit_log (
  id              BIGSERIAL PRIMARY KEY,
  user_id         TEXT NOT NULL,    -- Clerk user ID or 'system'
  action          TEXT NOT NULL,    -- 'score' | 'override_tier' | 'activate' | ...
  entity_type     TEXT NOT NULL,    -- 'account' | 'signal' | ...
  entity_id       BIGINT NOT NULL,
  details         JSONB,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Index hot paths
CREATE INDEX idx_signals_account_observed ON signals(account_id, observed_at DESC);
CREATE INDEX idx_signals_pending ON signals(processed_at) WHERE processed_at IS NULL;
CREATE INDEX idx_signals_triage ON signals(observed_at DESC);
CREATE INDEX idx_runs_account_created ON scoring_runs(account_id, created_at DESC);
CREATE INDEX idx_accounts_tier ON accounts(current_tier);
CREATE INDEX idx_accounts_activated ON accounts(is_activated) WHERE is_activated;
CREATE INDEX idx_audit_user_action ON audit_log(user_id, action, created_at DESC);
```

Signal types (enum strings)

`leadership_change`, `funding_round`, `job_posting_rcm`, `job_posting_ops`, `press_release`, `expansion_news`, `ehr_change`, `ma_announcement`, `earnings_miss`, `layoff_announcement`,

`competitor_mention`, `conference_speaking`, `ad_engagement`, `website_visit`.

Signal sources (enum strings)

`definitive`, `apollo`, `apify_linkedin`, `crunchbase`, `google_news_rss`, `linkedin_ads_api`, `manual`, `web_search`.

Domain logic

Scoring flow (single account)

1. **Trigger**: manual UI submit | signal-driven re-score | scheduled batch
2. **Job enqueued**: Arq job `score_account(account_id, segment, triggered_by, triggering_signal_id?)`
3. **Worker**: loads prompt → calls Claude Opus 4.7 with `web_search` → streams tokens (for UI live view)
4. **Parse**: extract structured JSON block, validate against Pydantic `ScoredAccount`. If parse fails, save with `parse_failed=TRUE` and `raw_markdown` only.
5. **Persist**: insert `scoring_runs` row; update `accounts.current_score/tier/last_scored_at`; log to `audit_log`.
6. **PDF**: generated on demand from `raw_markdown` (markdown → HTML → Chrome headless → PDF). Not stored — regenerable.

Re-scoring trigger policy

```
when signal arrives:
  if account.current_tier in {'Tier 1', 'High Fit', 'Tier 2', 'Medium Fit'}:
    if no scoring_run in last 24h:
      enqueue score_account(triggered_by='signal')
    else:
      mark signal pending (will batch in nightly job)
  else:
    log signal, no re-score
```

Rate limit: max 1 auto-re-score per account per 24h. Manual re-score is unlimited.

Tier override

Galyna's override is **never** written back into `scoring_runs.tier`. Original score is preserved; override lives in `tier_overrides` with the active row pointing to the latest scoring run. UI shows: "Claude: Medium Fit · Galyna: High Fit (note: 'PE backing not weighted enough')".

Cost tracking

Every Claude call logs `input_tokens`, `output_tokens`, computed `cost_usd` to `scoring_runs`. Dashboard tile shows MTD spend. Alert via Sentry when MTD > $100 (configurable env var).

UI / UX spec (brief Claude Design with this)

Three primary screens. Side nav: Triage · Accounts · Score · Settings.

1. Triage Dashboard (default landing)

Purpose: "What changed since I last looked?"

- **Hero strip**: 4 stats — New Tier 1 this week · Score-ups this week · Score-downs · Unprocessed signals
- **Activity feed** (chronological, last 7 days, paginated):
- Each row: account name (link) · signal type icon · 1-line summary · source badge · time ago
- Inline actions: View account · Dismiss · Mark reviewed
- **Filter bar**: segment chips, signal type chips, tier chips, source chips
- **Empty state**: "Nothing new. Time to score someone new" (links to Score-New)

2. Score-New form

Purpose: Galyna types a company, gets a scored report.

- Form: `Company name` (text) + `Segment` (3-button toggle: Specialties / Payer / Health System) + Submit
- On submit: form replaced by live scoring stream view (the same UI panels the saved report will have, populated as Claude streams)
- Progress strip at top: ⏳ Searching the web → 🧠 Analyzing → ✅ Done
- After completion: "Save to library" + "Download PDF" buttons

3. Account Detail page

Purpose: the canonical view of one account.

URL: `/accounts/[id]`

Tab 1 — One-pager (default): renders the latest scoring run's markdown. Adds two new panels Galyna asked for: - **Signal timeline** (chronological, all signals for this account, expandable rows) - **Sources panel** (every URL Claude cited, deduplicated, grouped by domain)

Tab 2 — History: list of every past `scoring_runs` row for this account. Click to view that run. (Phase 1 has the data; UI in Phase 5.)

Tab 3 — Override: form to set tier override. Dropdown of tier names · text area for reason · Save. Audit-logged.

Top-right actions: Re-score now · Download PDF · Mark activated/deactivated.

Production-grade requirements (the "best practices" non-negotiables)

Category	Practice	Tool
Repo structure	Monorepo: `apps/api` (FastAPI) + `apps/web` (Next.js) + `packages/shared` (Pydantic types → TS via openapi-typescript)	pnpm workspaces + uv
Type safety	Pydantic v2 at every API boundary; OpenAPI → TS client; strict mode on both sides	Pydantic, openapi-typescript
Testing	Unit (pytest), API integration (httpx + testcontainers Postgres), component (Vitest), smoke E2E (Playwright on critical paths only)	pytest, vitest, playwright
Linting + format	Ruff (Python), Biome (JS/TS). Both pre-commit + CI.	ruff, biome
Type checking	mypy strict on Python; tsc strict on TS	mypy, tsc
Pre-commit	Hooks for: ruff, mypy, biome, secret-scan	pre-commit
CI/CD	GitHub Actions: lint → type → test → build on every PR; auto-deploy `main` → Railway	GH Actions
DB migrations	Alembic with autogenerate + manual review; one migration per PR	Alembic
Secrets	Railway env vars (V1); upgrade path to Doppler if multi-env	Railway secrets
Auth	Clerk; protect every route except `/health`; audit log on every state-changing action	Clerk + middleware
Error tracking	Sentry for unhandled exceptions on api + web	Sentry
Structured logging	JSON logs with trace IDs; sent to Railway log drain	`structlog` (py) + `pino` (node)
Rate limiting	Per-endpoint (`fastapi-limiter`); per-LLM-call cost cap (refuse if MTD > budget)	fastapi-limiter
API versioning	`/api/v1/...` from day 1	FastAPI router
Background jobs	Arq with retries (3), exponential backoff, dead-letter handling, job-level cost budget	Arq

Category	Practice	Tool
Idempotency	All POST endpoints accept `Idempotency-Key` header; Arq jobs idempotent by deterministic job IDs	Custom middleware
Health + readiness	`/health` (liveness), `/ready` (DB + Redis + Anthropic connectivity check)	FastAPI routes
Feature flags	Simple env-var-driven flag dict; revisit LaunchDarkly only if needed	Custom
DB backups	Railway-managed daily snapshots, 7-day retention	Railway
Cost guardrails	Per-run cost logged; MTD alert at $100; hard cap at $500 (env-configurable)	Custom + Sentry
Docs	OpenAPI auto-generated; README with architecture diagram + runbook	FastAPI + handwritten
Repo hygiene	Conventional commits; squash-merge PRs; CHANGELOG via release-please	release-please

Integration map

Source	Mechanism	Signal types	Phase
Anthropic API	REST + streaming	(engine, not signal)	1 (done)
Definitive Health	REST API	account_seed, ehr_change	2
Apollo	REST API	leadership_change, contact_enrichment	3
Apify (LinkedIn)	Apify SDK	job_posting_rcm, profile_update, competitor_mention	4
Crunchbase	REST API	funding_round, ma_announcement	5
Google News RSS	RSS feeds + parsing	press_release, expansion_news, layoff_announcement	5
LinkedIn Ads API	REST API	ad_engagement	6
Slack	Webhook (outgoing)	(V1: none; V2: tier-1 alerts)	post-V1

Source	Mechanism	Signal types	Phase
Salesforce	REST API	(V2: account sync + ABM touches)	post-V1

Phase plan (concrete, post-this-doc)

#	Phase	Outcome	Effort
1	**Foundation refactor**	Modular `abm_scorer/` package, Pydantic models, Postgres schema via Alembic, Arq worker, structured logging	2 sessions
2	**Web app skeleton**	Next.js + Clerk auth + FastAPI scaffold + monorepo + CI + Sentry; deploy to Railway	2 sessions
3	**Definitive ingestion + Triage**	Definitive connector, accounts seeded, basic triage view live	2 sessions
4	**Score-New flow + Account Detail**	Galyna can score new accounts in-browser and view per-account one-pagers; PDF download; tier override	2 sessions
5	**Apollo connector**	Leadership change signals flow in; auto-rescore Tier 1/2 wires up end-to-end	1 session
6	**Apify connector**	LinkedIn job posts + profile updates streaming in	1 session
7	**Crunchbase + Google News**	Funding + press release signals	1 session
8	**LinkedIn Ads + polish**	Ad engagement signals; cost dashboard tile; backup automation; runbook	1 session

Total: ~12 sessions to reach V1-complete. Each phase ships independently.

QA / validation (placeholder, designed open-ended)

We don't yet know the dominant failure modes. The data model supports the following audits without code changes:

- **Score drift over time** — `scoring_runs` history lets us detect a Tier 1 account suddenly scoring 18.
- **Override frequency by Galyna** — `tier_overrides` aggregated per segment tells us where the prompt is systematically off.
- **Parse failure rate** — `scoring_runs.parse_failed=TRUE` % per week.
- **Signal noise** — signals that never lead to a tier change after re-score.

Phase 9 (post-V1) builds dashboards on top of these. Phase 1 just makes sure the data is there to support them.

Open questions / dependencies before kickoff

Item	Owner	Blocks
Definitive Health API key + endpoint docs	Sunny / Galyna to confirm	Phase 3
Apollo API access (Magical org account?)	Galyna	Phase 5
Apify account + LinkedIn target queries	Sunny	Phase 6
Crunchbase API access	TBD	Phase 7
Anthropic API key under Magical org (not Sunny's personal)	Galyna / IT	Operational
Clerk free-tier signup + Google Workspace SSO config	Sunny	Phase 2
Railway project upgrade (free → starter if needed)	Sunny	Phase 2
Sentry free account	Sunny	Phase 2
Domain (e.g. `abm.magical.com`) — needed or `*.railway.app` is fine?	Galyna	Phase 2
Initial seed data — Definitive bulk or Galyna's existing Google Sheet?	Galyna	Phase 3

Glossary (for Claude Design + onboarding)

- **Account**: a company we're targeting (e.g. OrthoIndy). One segment. One row in `accounts`.
- **Contact**: a person at an account (e.g. CEO). Multiple per account.

- **Signal**: an observed event tied to an account (e.g. "new CFO hired Oct 2025"). Drives re-scoring + triage.
- **Scoring Run**: one execution of Claude scoring. Audited, versioned.
- **Tier**: bucketed scoring outcome — "Tier 1", "High Fit", etc. Segment-dependent.
- **Override**: Galyna's manual tier adjustment on top of Claude's score. Auditable, not destructive.
- **Activation**: marking an account as "ready for outbound." In V1, just a flag. In V2, triggers sequences.
- **Triage**: the daily/weekly review surface — what's new, what changed.
- **One-pager**: the per-account structured report (the markdown + PDF). The canonical deliverable.

Appendix A — Repo layout (target after Phase 2)

```
abm-scorer/                          # monorepo root
├── pnpm-workspace.yaml
├── pyproject.toml                   # workspace root (uv)
├── .github/workflows/
│   ├── ci.yml                       # lint + type + test on PR
│   └── deploy.yml                   # main → Railway
├── apps/
│   ├── api/                         # FastAPI service
│   │   ├── pyproject.toml
│   │   ├── alembic/
│   │   ├── src/abm_api/
│   │   │   ├── main.py              # FastAPI app
│   │   │   ├── config.py
│   │   │   ├── models.py            # Pydantic
│   │   │   ├── db.py                # SQLAlchemy engine + session
│   │   │   ├── routers/             # /accounts /signals /scoring /overrides
│   │   │   ├── services/            # scoring, ingestion, triage logic
│   │   │   ├── connectors/          # definitive, apollo, apify, ...
│   │   │   ├── workers/             # Arq job definitions
│   │   │   ├── prompts/             # txt files (carried over from V1)
│   │   │   └── output.py            # normalize_table_cells, pdf render
│   │   └── tests/
│   ├── web/                         # Next.js app
│   │   ├── package.json
│   │   ├── app/                     # App Router
│   │   │   ├── (auth)/sign-in
│   │   │   ├── (app)/triage
│   │   │   ├── (app)/accounts/[id]
│   │   │   ├── (app)/score-new
```

```
│   │   │   └── api/                    # any Next.js route handlers
│   │   ├── components/                 # shadcn/ui + custom
│   │   ├── lib/                        # api client (generated from OpenAPI)
│   │   └── tests/
│   └── worker/                         # Arq worker entry (uses apps/api code)
├── packages/
│   └── shared-types/                   # generated TS types from API OpenAPI
├── docs/
│   ├── V1_VISION.md                    # this file
│   ├── ABM_PROJECT_ARCHITECTURE.md     # the broader 7-box doc
│   ├── RUNBOOK.md                      # incident response, common ops
│   └── DECISIONS.md                    # ADRs for non-obvious calls
├── scripts/
│   ├── seed_demo_accounts.py
│   └── backup_db.sh
└── .env.example
```

Appendix B — Architectural Decision Records to write (post-vision)

Short ADRs to commit so future-you remembers *why*:

1. Postgres over SQLite (V1 → V2 portability)
2. Arq over Celery (async-native, lighter)
3. Clerk over self-rolled auth (multi-user upgrade path)
4. Monorepo over polyrepo (shared types, single deploy)
5. uv + ruff + biome (Astral stack + Rust tooling)
6. One segment per account (vs many-to-many)
7. PDFs regenerated on demand (not stored)
8. Tier override appended, never overwritten (audit integrity)
9. Auto-rescore only Tier 1/2 (cost vs. signal latency tradeoff)
10. Notion sync explicitly removed (app is source of truth)

What to do next

1. **Galyna sign-off**: walk her through this doc; confirm priorities and segments
2. **Brief Claude Design**: paste the *UI / UX spec* section into Claude Design / Artifacts; get mocks of the 3 screens
3. **Start Phase 1 implementation**: see ABM_PROJECT_ARCHITECTURE.md → Phase 1 steps
4. **Open issues**: convert "Open questions / dependencies" into a tracking list (GitHub issues or Notion)